DBA Learnings

CockroachDB

Complete Technical Knowledge Base

Distributed SQL · Architecture · Operations · HA/DR · Interview Prep

Senior Technical Architect & Documentation Reference

Aligned to CockroachDB v25.x / v26.x (self-hosted)

sqldbachamps.com

Table of Contents

1. Overview 3
2. Architecture 4
3. Components 7
4. Installation 8
5. Configuration 10
6. Prerequisites 11
7. Security Best Practices 12
8. Performance Tuning 13
9. Monitoring 14
10. Logging 16
11. Backup & Recovery 17
12. Disaster Recovery 18
13. High Availability 19
14. Troubleshooting 22
15. Common Errors 23
16. FAQs 24
17. Interview Questions 25
18. Real-world Use Cases 26
19. Best Practices 26
20. References 27

1. Overview

CockroachDB is a **distributed SQL (NewSQL) database** developed by Cockroach Labs. It presents a single logical SQL database to applications while transparently sharding, replicating, and rebalancing data across many nodes. Its design goal is captured by its name: like a cockroach, the cluster is built to **survive** the loss of disks, nodes, availability zones, and even whole regions without data loss and with minimal disruption.

It speaks the **PostgreSQL wire protocol** and a large subset of PostgreSQL SQL, so most Postgres drivers, ORMs, and tools connect with little or no change. Under the hood it is nothing like a single-node Postgres: every row lives in a replicated, consensus-backed key-value range.

1.1 Key Characteristics

Property	What CockroachDB Provides
Data model	Relational SQL (PostgreSQL-compatible wire protocol & dialect)
Consistency	Strong / serializable isolation by default; no dirty or non-repeatable reads
Scalability	Horizontal — add nodes to increase throughput & capacity, near-linear
Availability	Survives node / AZ / region failure via Raft consensus (RF ≥ 3)
Transactions	Distributed, multi-statement, ACID across ranges and nodes
Geo-distribution	Multi-region tables, data domiciling, survival goals, follower reads
Operations	Automatic sharding, rebalancing, and self-healing — no manual sharding
Engine	Pebble (Go LSM key-value store); Raft for replication

1.2 Where It Fits

- OLTP systems that must scale beyond a single primary and cannot tolerate downtime.
- Global applications needing data close to users while keeping a single logical database.
- Workloads migrating off sharded MySQL/Postgres or expensive proprietary clustered databases.
- Regulated workloads needing row-level data domiciling (GDPR-style residency).

> **NOTE**
> CockroachDB targets OLTP and HTAP-leaning workloads. It is not a data warehouse — for heavy analytical scans, pair it with a columnar system (e.g., Snowflake) via CDC/changefeeds rather than running large aggregations directly.

1.3 Licensing & Editions (important — changed in late 2024)

CockroachDB's licensing changed materially. It began under Apache 2.0, moved to the Business Source License (BSL) in 2019, and in **November 2024 (with v24.3)** Cockroach Labs retired the free, open-source "Core" edition and consolidated on a single **CockroachDB Enterprise** offering under the proprietary, source-available **CockroachDB Software License**.

Edition	Cost	Who it is for	Notes
Enterprise Free	Free (annual renewal)	Individuals, students, researchers, businesses under ~$10M revenue	Full feature set; telemetry is mandatory (cannot be disabled)
Enterprise Trial	Free, 30 days	Evaluation	Community support only
Enterprise (paid)	Annual contract	Companies over the revenue threshold & government	Full support & SLAs; telemetry can be disabled
CockroachDB Cloud	Usage / subscription	Managed service users	Basic, Standard, Advanced plans; Cockroach Labs operates it

> **WARNING**
> The source is still viewable but is no longer free/open-source in the OSI sense. Self-hosted deployments require a license key (even the free one), and the free tier forces telemetry. Confirm your organisation's revenue threshold and telemetry posture before production use.

1.4 Versioning

Releases follow calendar versioning **vYY.R.PP** — year, major (1–4 per year), patch. Major versions ship roughly quarterly, alternating **Regular** (long support windows, can be promoted to LTS) and **Innovation** (shorter support, newest features). At the time of writing the current line is the v25.x / v26.x family.

2. Architecture

CockroachDB is organised as a stack of five logical layers. A SQL statement enters at the top and is progressively translated into replicated key-value operations at the bottom. Every node runs the **entire stack** — there are no special "master" or "coordinator" node types, which is what makes the cluster symmetric and self-healing.

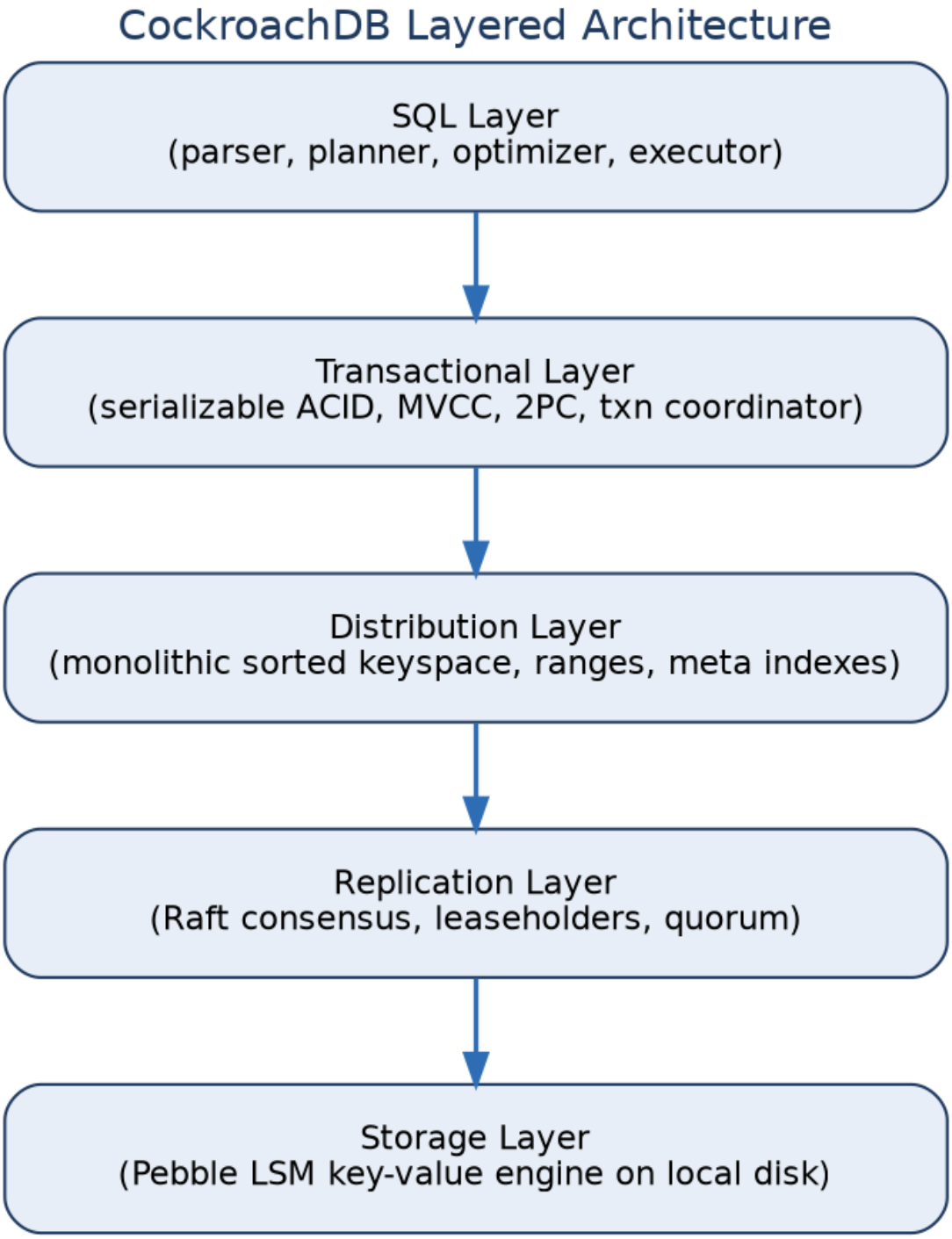


Figure 2.1 — The five architectural layers of CockroachDB.

Mermaid source (editable / re-renderable):

```
flowchart TD
  A["SQL Layer<br/>parser, planner, optimizer, executor"] -->
  B["Transactional Layer<br/>serializable ACID, MVCC, 2PC"] -->
  C["Distribution Layer<br/>monolithic sorted keyspace, ranges"] -->
  D["Replication Layer<br/>Raft consensus, leaseholders, quorum"] -->
  E["Storage Layer<br/>Pebble LSM engine on local disk"]
```

2.1 Layer Responsibilities

Layer	Responsibility
SQL	Parses SQL, builds & cost-optimizes the plan, executes it, and maps rows to/from key-value pairs. Presents the PostgreSQL interface.
Transactional	Provides serializable, distributed ACID transactions using MVCC, write intents, and an atomic commit protocol.
Distribution	Treats all data as one giant sorted key-space, split into ~512 MiB ranges; routes each key to the range that owns it via meta indexes.
Replication	Keeps each range consistently replicated (default 3 copies) using the Raft consensus protocol; elects a leaseholder per range.
Storage	Persists key-value data on each node using Pebble, a RocksDB-style LSM engine, with a write-ahead log.

2.2 Cluster Topology

Nodes form a peer-to-peer mesh. They exchange cluster metadata (node liveness, range locations, capacity) through a gossip protocol, so any node can act as the SQL gateway for any client and route requests to wherever the data lives.

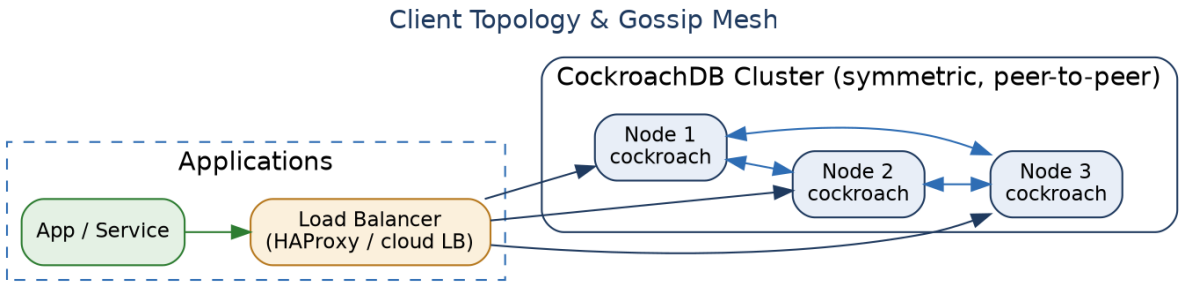


Figure 2.2 — Symmetric cluster with a client load balancer and gossip mesh.

Mermaid source (editable / re-renderable):

```
flowchart LR
  app[App / Service] --> lb[Load Balancer]
  lb --> n1[Node 1]
  lb --> n2[Node 2]
  lb --> n3[Node 3]
  n1 <--> n2
  n2 <--> n3
  n1 <--> n3
```

2.3 Ranges, Replicas & Leaseholders

Data is split into **ranges** (contiguous key spans, ~512 MiB). Each range is replicated to multiple nodes as **replicas**. One replica is elected the **leaseholder**: it coordinates all reads and writes for that range and is (usually) also the Raft leader. This is the unit of both distribution and fault tolerance.

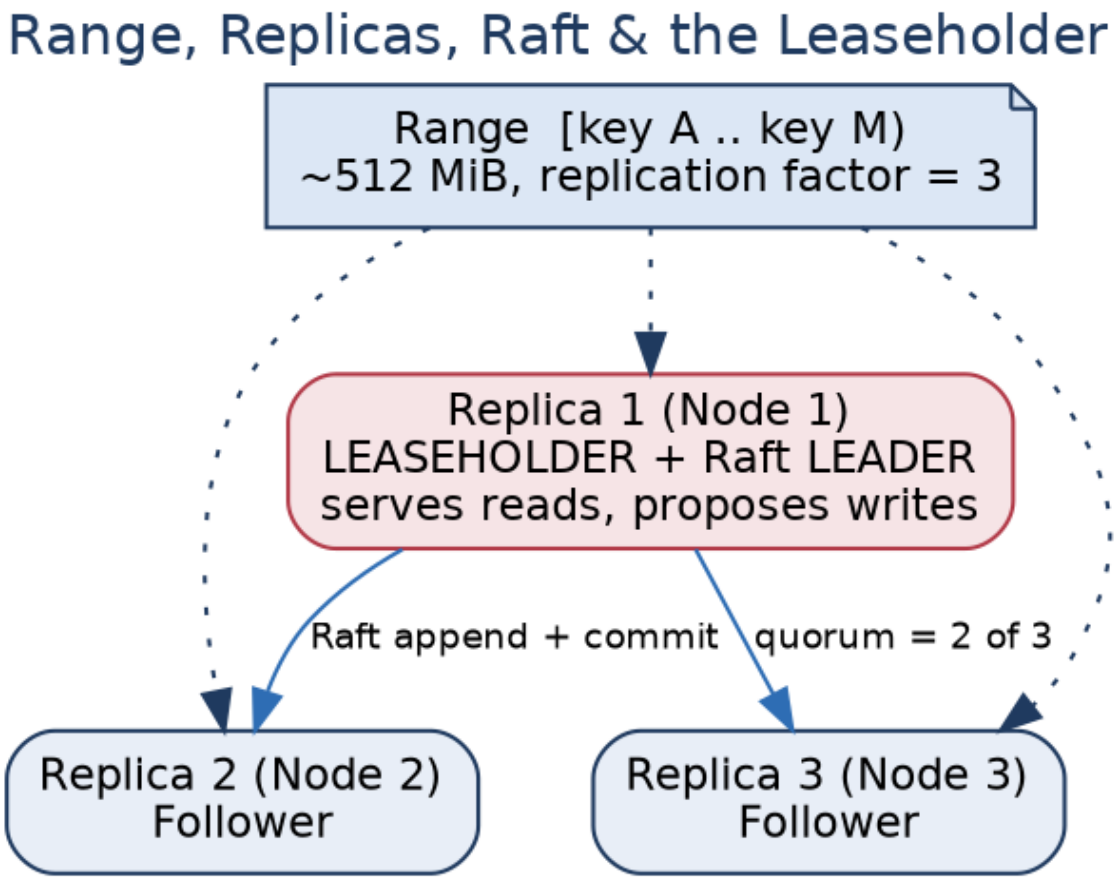


Figure 2.3 — A single range, its three replicas, and the leaseholder.

Mermaid source (editable / re-renderable):

```
flowchart TD
  R["Range [A..M)  RF=3"] -.-> LH["Replica 1 — LEASEHOLDER + Raft leader"]
  R -.-> F1["Replica 2 — Follower"]
  R -.-> F2["Replica 3 — Follower"]
  LH -- "Raft append + commit (quorum 2/3)" --> F1
  LH -- "Raft append + commit" --> F2
```

> **BEST PRACTICE**
> Keep the replication factor odd (3 or 5). An odd number tolerates the loss of ⌊(n−1)/2⌋ replicas while keeping a quorum — 3 tolerates 1 failure, 5 tolerates 2 — without wasting capacity on a copy that adds no additional fault tolerance.

3. Components

A CockroachDB deployment is deliberately simple to operate: a single self-contained binary plus your storage and (optionally) a client load balancer. The "components" below are mostly logical roles inside that binary rather than separate services to install.

Component	Type	Role
`cockroach` binary	Executable	One static Go binary that is server, CLI, and tools in one. Every node runs the same binary.
Node	Process	A single running `cockroach` instance. Holds replicas, serves SQL, participates in Raft & gossip.
Store	Disk	A storage device backing a node (`--store`). A node can have multiple stores; each holds ranges.
Range	Data unit	A ~512 MiB contiguous slice of the keyspace; the unit of replication & rebalancing.
Replica	Data copy	One physical copy of a range on a store. RF copies exist per range.
Leaseholder	Role	The replica that serves reads and sequences writes for a range.
Raft group	Protocol	The consensus group formed by a range's replicas; commits writes by quorum.
Gossip	Protocol	Peer-to-peer dissemination of cluster metadata (liveness, range locations, capacity).
Gateway node	Role	The node a client connects to; it plans the query and coordinates the transaction.
DB Console	UI	Built-in web UI on port `8080` for metrics, schema, jobs, and cluster health.
Pebble	Engine	The embedded LSM key-value storage engine (successor to RocksDB).
Distributed SQL (DistSQL)	Engine	Pushes computation (filters, joins, aggregations) to the nodes holding the data.

TIP

Because it is a single binary with no external dependencies (no ZooKeeper, no separate config store, no external coordinator), CockroachDB is straightforward to run in containers and Kubernetes. Operational complexity lives in topology and tuning, not in wiring services together.

4. Installation

CockroachDB installs as a single binary. Below are the four most common paths: local binary, a secure multi-node cluster, Docker, and Kubernetes via the Operator/Helm.

4.1 Linux binary (single node, insecure — local dev only)

```
# Download & install the binary (adjust version as needed)
curl https://binaries.cockroachdb.com/cockroach-v25.4.8.linux-amd64.tgz | tar -xz
sudo cp -i cockroach-v25.4.8.linux-amd64/cockroach /usr/local/bin/

# Verify
cockroach version

# Start a single-node dev server (INSECURE — never in production)
cockroach start-single-node --insecure --listen-addr=localhost:26257 \
```

```
  --http-addr=localhost:8080 --background

# Connect with the built-in SQL shell
cockroach sql --insecure --host=localhost:26257
```

4.2 Secure multi-node cluster (production shape)

Production clusters run with TLS. Generate a CA, then per-node and client certificates, distribute them, start each node pointing at its peers, then initialize the cluster once.

```
# 1) Create the CA and certificates (run on a secure admin host)
cockroach cert create-ca --certs-dir=certs --ca-key=my-safe-directory/ca.key

# One node cert per node — list every name/IP the node is reachable by
cockroach cert create-node node1.example.com 10.0.0.1 localhost 127.0.0.1 \
  --certs-dir=certs --ca-key=my-safe-directory/ca.key
# (repeat create-node for node2, node3 ... with their own names/IPs)

# A client cert for the root user
cockroach cert create-client root --certs-dir=certs --ca-key=my-safe-directory/ca.key

# 2) Start each node (run on each host; --join lists 3-5 seed nodes)
cockroach start --certs-dir=certs \
  --advertise-addr=10.0.0.1 \
  --join=10.0.0.1,10.0.0.2,10.0.0.3 \
  --cache=.25 --max-sql-memory=.25 --background

# 3) Initialize the cluster ONCE, from any node
cockroach init --certs-dir=certs --host=10.0.0.1
```

WARNING

Run `cockroach init` exactly once for the whole cluster. Running it again, or on a second set of nodes, creates a separate cluster. Set --cache and --max-sql-memory explicitly (e.g. .25 each) — the defaults are conservative and will bottleneck a dedicated host.

4.3 Docker

```
# Pull the multi-platform image (Intel + ARM)
docker pull cockroachdb/cockroach:v25.4.8

# Single-node dev container
docker run -d --name=roach1 --hostname=roach1 \
  -p 26257:26257 -p 8080:8080 \
  -v "roach1-data:/cockroach/cockroach-data" \
  cockroachdb/cockroach:v25.4.8 start-single-node --insecure

# SQL shell into it
docker exec -it roach1 ./cockroach sql --insecure
```

4.4 Kubernetes

For Kubernetes, use the **CockroachDB Kubernetes Operator** or the official Helm chart. The Operator manages a StatefulSet, PersistentVolumeClaims, certificates, and safe rolling upgrades.

```
# Helm (quick path)
helm repo add cockroachdb https://charts.cockroachdb.com/
helm repo update
helm install crdb cockroachdb/cockroachdb \
  --set statefulset.replicas=3 \
  --set storage.persistentVolume.size=100Gi \
  --set tls.enabled=true

# Check pods & initialize (the chart runs a cluster-init Job automatically)
kubectl get pods -l app.kubernetes.io/name=cockroachdb
```

> **BEST PRACTICE**
> Use local SSD or a fast block-storage class with a dedicated PVC per pod. Never place two nodes' stores on the same physical disk — you lose the fault-isolation Raft depends on.

5. Configuration

CockroachDB is configured in three places: (1) start-up flags on the binary, (2) cluster settings applied via SQL that affect the whole cluster, and (3) zone configurations / SQL schema that control replication and placement per object.

5.1 Key start-up flags

Flag	Purpose	Typical value
`--store`	Data directory / device for a store	`/mnt/data1` (dedicated SSD)
`--join`	Seed nodes used to discover the cluster	3–5 stable node addresses
`--advertise-addr`	Address peers & clients use to reach this node	Routable host/IP:26257
`--locality`	Region/zone tags for placement & survival goals	`region=us-east,zone=a`
`--cache`	Storage engine block cache	`.25` (25% of RAM) on dedicated hosts
`--max-sql-memory`	Memory budget for SQL execution	`.25` of RAM
`--certs-dir`	TLS certificate directory (secure mode)	`certs`

5.2 Cluster settings (SQL)

```
-- Inspect and change cluster-wide behaviour
SHOW ALL CLUSTER SETTINGS;

-- Enterprise license (required for self-hosted, even the free tier)
SET CLUSTER SETTING cluster.organization = 'Acme Inc';
SET CLUSTER SETTING enterprise.license = 'xxxx-your-license-key-xxxx';
```

```
-- Statement diagnostics & slow-query logging
SET CLUSTER SETTING sql.log.slow_query.latency_threshold = '200ms';

-- Keep 25 hours of time-travel for AS OF SYSTEM TIME / incremental backups
SET CLUSTER SETTING gc.ttlseconds = 90000;   -- per zone; see below
```

5.3 Replication & placement (zone configs)

Zone configurations control how many replicas an object has, where they live, and how long old MVCC versions are retained (the GC TTL). They can be set per database, table, index, or partition.

```
-- Increase resilience of a critical table to 5 replicas
ALTER TABLE payments CONFIGURE ZONE USING num_replicas = 5;

-- Pin a table's data to a region for residency
ALTER TABLE eu_customers CONFIGURE ZONE USING
  constraints = '[+region=eu-west]';

-- Shorten GC TTL on a hot table to reduce storage of old versions
ALTER TABLE sessions CONFIGURE ZONE USING gc.ttlseconds = 600;

-- Multi-region declarative syntax (preferred for geo apps)
ALTER DATABASE app SET PRIMARY REGION 'us-east';
ALTER DATABASE app ADD REGION 'eu-west';
ALTER DATABASE app SURVIVE REGION FAILURE;
```

> **NOTE**
> GC TTL is a trade-off: a longer TTL enables longer point-in-time recovery windows and follower reads further back, but retains more historical versions on disk. The 25-hour default supports daily+incremental backup patterns.

6. Prerequisites

6.1 Hardware (per node, production)

Resource	Minimum	Recommended	Notes
vCPU	4	8–16	Scale SQL throughput; avoid oversubscription
RAM	8 GB	32–64 GB	Give ~25% to cache and ~25% to SQL memory
Storage	SSD	Local NVMe SSD	Low-latency, high-IOPS; one store per device
Store size	—	≤ 2.5 TB per node	Smaller stores rebalance & recover faster
Network	1 Gbps	10 Gbps, <10 ms intra-region	Raft is latency-sensitive

> **WARNING**
> Never run production on network-attached storage with unpredictable latency or on burstable CPU instances. Raft quorum writes are on the critical path — storage and CPU throttling turn into user-visible tail latency and, in the worst case, lease thrashing.

6.2 Cluster sizing

- Minimum 3 nodes for any resilience (survives 1 node failure at RF=3).
- Spread nodes across at least 3 fault domains (AZs) so an AZ loss keeps a quorum.
- For region survival, use ≥ 3 regions so a full region loss still leaves a quorum.

6.3 Operating system & runtime

Requirement	Detail
OS	64-bit Linux (production); macOS/Windows for dev only
Clock sync	NTP/chrony with low offset — clock skew beyond max-offset (default 500 ms) ejects a node
File descriptors	Raise `ulimit -n` to at least 65,536
Filesystem	ext4 or xfs on dedicated SSD; disable atomic-write features that add latency
Architecture	x86-64 or ARM64 (multi-platform images available)

6.4 Network ports

Port	Purpose	Exposure
`26257`	SQL client connections + inter-node RPC	App tier + all nodes
`8080`	DB Console (HTTP/HTTPS) & Prometheus metrics endpoint	Operators / monitoring only

BEST PRACTICE

Put a TCP load balancer (HAProxy, or a cloud L4 LB) in front of port 26257 so clients need not know individual node addresses and so a failed node is removed from rotation automatically.

7. Security Best Practices

CockroachDB security spans transport encryption, authentication, authorization, encryption at rest, auditing, and network isolation. Treat "insecure mode" as a dev-only convenience — production must be secure end to end.

7.1 Transport encryption (TLS)

- Always run with `--certs-dir`; every node↔node and client↔node connection is mutually authenticated with certificates.
- Rotate node and CA certificates before expiry; CockroachDB supports online rotation without downtime.

```
# Rotate a node certificate, then reload without restart
cockroach cert create-node node1 10.0.0.1 --certs-dir=certs --ca-key=ca.key
# On the node, signal a reload of certs:
kill -SIGHUP <cockroach_pid>
```

7.2 Authentication

- Prefer certificate authentication for services; use SCRAM-SHA-256 password auth for humans/tools that can't hold certs.
- Integrate with SSO/OIDC (GSSAPI/Kerberos and JWT/OIDC options in Enterprise) for centralized identity.

```
-- Create a least-privilege application role and user
CREATE ROLE app_rw;
GRANT SELECT, INSERT, UPDATE, DELETE ON DATABASE app TO app_rw;
CREATE USER svc_orders WITH PASSWORD 'use-a-secret-manager';
GRANT app_rw TO svc_orders;
```

7.3 Authorization (RBAC)

- Grant privileges to roles, not individual users; assign users to roles. Use the principle of least privilege.
- Reserve the `admin` role and `root` user for break-glass operations only.
- Use per-object/system privileges (e.g. `VIEWACTIVITY`, `CANCELQUERY`, `BACKUP`) instead of blanket admin.

7.4 Encryption at rest & secrets

- Enable store-level Encryption-At-Rest (Enterprise) with AES and regular key rotation, or rely on encrypted volumes (LUKS / cloud KMS-backed disks).
- Never embed passwords/keys in connection strings committed to source; use a secrets manager (Vault, AWS Secrets Manager, Azure Key Vault).

7.5 Auditing & network isolation

- Enable SQL audit logging on sensitive tables to capture access for SOX/PCI-DSS evidence.
- Place the cluster in a private subnet; expose only the load balancer and restrict the DB Console to operators via VPN/bastion.

```
-- Turn on audit logging for a sensitive table
ALTER TABLE cardholder_data EXPERIMENTAL_AUDIT SET READ WRITE;
```

> **BEST PRACTICE**
> Compliance mapping: TLS + cert auth (PCI-DSS Req 4 / 8), RBAC least privilege (Req 7), encryption at rest (Req 3), audit logging (Req 10, SOX access-tracking). Keep certificate inventory and rotation evidence — auditors ask for it.

8. Performance Tuning

Performance in CockroachDB is mostly about (1) good schema and index design, (2) keeping transactions short and contention-free, and (3) placing data close to where it is read. Because writes require Raft quorum, network latency between replicas is a first-order factor.

8.1 Transaction & MVCC lifecycle

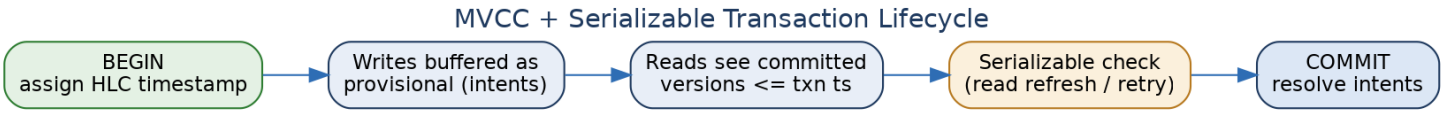


Figure 8.1 — Serializable transaction lifecycle; retries can occur at the serializable check.

Mermaid source (editable / re-renderable):

```
flowchart LR
  B[BEGIN — assign HLC ts] --> W[Writes buffered as intents]
  W --> R[Reads see versions <= ts]
  R --> C[Serializable check / refresh]
  C --> M[COMMIT — resolve intents]
```

8.2 Schema & indexing

- Design primary keys to spread writes. Monotonic keys (auto-increment, timestamps) create a single hot range; prefer UUID or hash-sharded indexes for high-insert tables.
- Use covering / STORING indexes so lookups need only one index, avoiding a second range read.
- Interleaving is removed; instead co-locate related data with multi-region and partitioning features.

```
-- Spread hot inserts across ranges with a hash-sharded index
CREATE TABLE events (
  id        UUID DEFAULT gen_random_uuid(),
  ts        TIMESTAMPTZ DEFAULT now(),
  payload   JSONB,
  PRIMARY KEY (id)
);
CREATE INDEX ON events (ts) USING HASH WITH (bucket_count = 8);

-- Covering index avoids a second read
CREATE INDEX ON orders (customer_id) STORING (status, total);
```

8.3 Reducing contention & latency

- Keep transactions short; long-running txns hold intents and cause serialization retries.
- Handle 40001 (serialization) retries in the client with exponential backoff — expected under contention.
- Batch inserts and use `UPSERT` instead of read-then-write where possible.
- Use follower reads (`AS OF SYSTEM TIME follower_read_timestamp()`) for read-heavy, slightly-stale queries to offload leaseholders and read locally.

```
-- Stale but local & cheap read (served by nearest follower)
SELECT * FROM catalog
AS OF SYSTEM TIME follower_read_timestamp()
WHERE region = 'eu-west';
```

8.4 Diagnose with EXPLAIN & statement stats

```
EXPLAIN ANALYZE (VERBOSE) SELECT ... ;      -- real execution plan + timings
SELECT * FROM crdb_internal.node_statement_statistics
ORDER BY service_lat_avg DESC LIMIT 20;      -- worst statements by latency
```

> **TIP**
> The DB Console → SQL Activity page surfaces top statements by latency, execution count, and contention, with plan details and index recommendations — start tuning there before hand-writing EXPLAINs.

9. Monitoring

CockroachDB exposes a rich Prometheus metrics endpoint and a built-in DB Console. The standard production stack scrapes metrics into Prometheus, visualizes them in Grafana, and pages via Alertmanager.

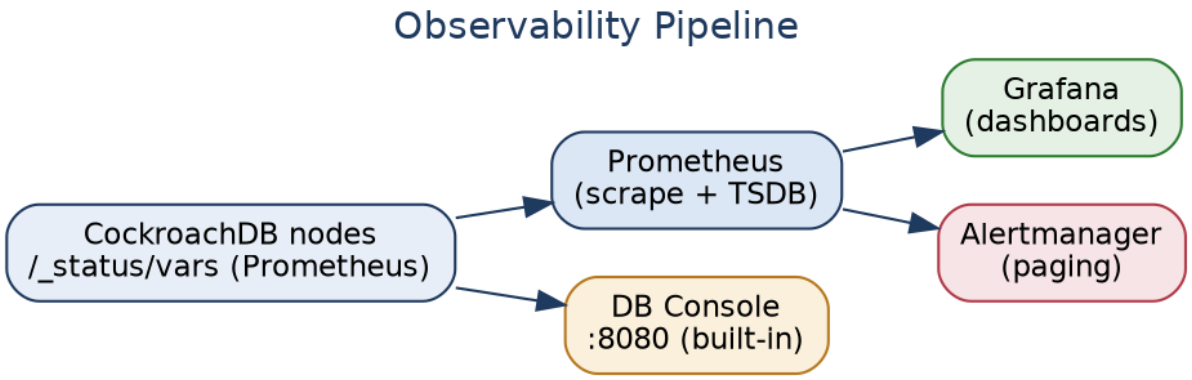


Figure 9.1 — Standard observability pipeline.

Mermaid source (editable / re-renderable):

```
flowchart LR
  crdb["CockroachDB nodes<br/>/_status/vars"] --> prom[Prometheus]
  crdb --> dbc["DB Console :8080"]
  prom --> graf[Grafana]
  prom --> alert[Alertmanager]
```

9.1 Endpoints

- DB Console: `https://<node>:8080` — cluster overview, node map, hardware, SQL activity, jobs, ranges.
- Prometheus metrics: `https://<node>:8080/_status/vars` (per node).
- Health checks: `/health` (is the process up) and `/health?ready=1` (is the node ready to serve).

9.2 Key metrics to alert on

Metric / signal	Why it matters	Alert when
`liveness_livenodes`	Number of live nodes	Drops below expected count
`ranges_underreplicated`	Ranges below desired RF	> 0 for more than a few minutes
`ranges_unavailable`	Ranges with no quorum	> 0 (data unavailable — page immediately)
`sql_service_latency` p99	User-facing query latency	Sustained above SLO
`clock_offset_meannanos`	Clock skew vs peers	Approaching max-offset (500 ms)
`capacity_available`	Free store capacity	Below ~15–20%
`sys_cpu_combined_percent`	Node CPU saturation	Sustained > 80%
`rocksdb_read_amplification`	LSM read amplification	Rising trend (compaction debt)

> **CAUTION**
> `ranges_unavailable > 0` means some data cannot be served — a quorum is lost. This is a page-the-on-call event: identify the down nodes/AZ and restore quorum before it turns into a customer-visible outage.

10. Logging

CockroachDB has a structured, channel-based logging system. Each channel (OPS, HEALTH, SQL_EXEC, SQL_PERF, SQL_SCHEMA, AUDIT, etc.) can be routed to different sinks — files, stderr, network (Fluentd/Fluent Bit), or HTTP — with independent formats and filters.

10.1 Log channels

Channel	Contents
`OPS`	Operational events: node start/stop, decommission, membership changes
`HEALTH`	Node health, liveness, resource pressure
`STORAGE`	Storage engine & replication events
`SQL_SCHEMA`	DDL / schema change activity
`SQL_EXEC`	Statement execution (incl. slow-query logging)
`SQL_PERF` / `SQL_INTERNAL_PERF`	Performance-related SQL events
`SENSITIVE_ACCESS` (AUDIT)	Audit log for sensitive-table access (compliance)
`SESSIONS`	Connection and authentication events

10.2 Configuring log sinks (YAML)

```
# logs.yaml — route channels to files with rotation, ship audit to network
file-defaults:
  dir: /var/log/cockroach
  max-file-size: 10MiB
  max-group-size: 100MiB
sinks:
  file-groups:
    ops:      { channels: [OPS, HEALTH] }
    sql-audit:
      channels: [SENSITIVE_ACCESS]
      auditable: true            # tamper-evident, never buffered
  fluent-servers:
    siem:
      address: fluent.internal:5170
      channels: [OPS, HEALTH, SENSITIVE_ACCESS]
      format: json-fluent
# Start with:  cockroach start --log-config-file=logs.yaml ...
```

BEST PRACTICE

Ship OPS, HEALTH, and AUDIT channels to a central SIEM. Mark audit sinks `auditable: true` so writes are synchronous and tamper-evident — essential for SOX/PCI-DSS evidence and for post-incident forensics.

10.3 Slow-query logging

```
SET CLUSTER SETTING sql.log.slow_query.latency_threshold = '250ms';
-- Statements slower than 250ms now emit to the SQL_PERF channel
```

11. Backup & Recovery

CockroachDB provides SQL-driven distributed backups to external object storage, with full and incremental backups, revision history, and point-in-time restore. Enterprise adds scheduled and managed backups.

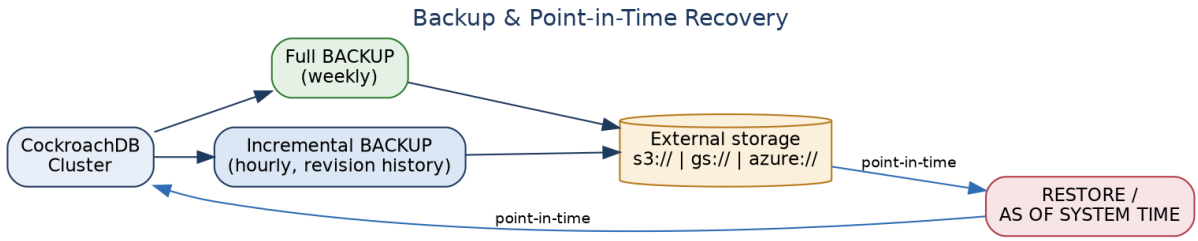


Figure 11.1 — Full + incremental backups to object storage with point-in-time restore.

Mermaid source (editable / re-renderable):

```
flowchart LR
  cl[CockroachDB Cluster] --> full["Full BACKUP (weekly)"]
  cl --> inc["Incremental BACKUP (hourly)"]
  full --> store[(s3:// / gs:// / azure://)]
  inc --> store
  store --> rest["RESTORE / AS OF SYSTEM TIME"] --> cl
```

11.1 Taking backups

```
-- Full cluster backup with revision history to S3
BACKUP INTO 's3://acme-crdb-
backups/cluster?AWS_ACCESS_KEY_ID={id}&AWS_SECRET_ACCESS_KEY={secret}'
  AS OF SYSTEM TIME '-10s'
  WITH revision_history;

-- Incremental backup into the same collection (CRDB finds the base automatically)
BACKUP INTO LATEST IN 's3://acme-crdb-backups/cluster?...'
  WITH revision_history;

-- Database- or table-scoped backup
BACKUP DATABASE app INTO 's3://acme-crdb-backups/app?...';
```

11.2 Scheduled backups (Enterprise)

```
CREATE SCHEDULE app_backup
  FOR BACKUP INTO 's3://acme-crdb-backups/app?...'
  WITH revision_history
  RECURRING '@hourly'
  FULL BACKUP '@weekly'
  WITH SCHEDULE OPTIONS first_run = 'now';

SHOW SCHEDULES;   -- monitor status & last run
```

11.3 Restore & point-in-time recovery

```
-- Inspect a backup collection
SHOW BACKUPS IN 's3://acme-crdb-backups/cluster?...';

-- Restore the whole cluster to a specific point in time
RESTORE FROM LATEST IN 's3://acme-crdb-backups/cluster?...'
  AS OF SYSTEM TIME '2026-07-13 09:15:00';

-- Restore a single table into an existing database
RESTORE TABLE app.orders FROM LATEST IN 's3://acme-crdb-backups/app?...';
```

WARNING

Point-in-time restore only reaches back as far as your GC TTL (default 25h) unless the backup was taken `WITH revision_history`. For meaningful PITR windows, always use revision history and verify restores in a staging cluster on a schedule — an untested backup is a hope, not a recovery plan.

11.4 Logical export (portability)

```
-- Changefeeds stream row-level changes to Kafka / cloud sinks (Enterprise / CDC)
CREATE CHANGEFEED FOR TABLE orders
  INTO 'kafka://broker:9092'
```

```
WITH updated, resolved = '10s';
```

12. Disaster Recovery

CockroachDB folds a great deal of "disaster recovery" into its normal operation: because ranges are replicated by Raft across fault domains, losing a disk, node, or AZ is handled automatically without a separate failover procedure. DR planning focuses on the larger events — region loss, cluster corruption, human error, and cross-cluster replication.

12.1 Failure domains & what survives automatically

Failure	Survivable automatically?	Mechanism
Single disk / store	Yes	Replicas on other stores; range re-replicates
Single node	Yes (RF ≥ 3)	Quorum on surviving replicas; self-heal
Availability zone	Yes (nodes spread ≥ 3 AZs)	Quorum in other AZs
Whole region	Yes, if SURVIVE REGION FAILURE + ≥ 3 regions	Cross-region replicas keep quorum
Accidental DROP / bad deploy	Via PITR / backup restore	AS OF SYSTEM TIME, backup restore
Total cluster loss	Via backup restore to new cluster	RESTORE from external storage

12.2 RPO / RTO planning

- RPO (data loss window): near-zero for in-cluster failures (synchronous Raft). For total-loss scenarios, RPO = time since last incremental backup — so back up frequently (hourly or better).
- RTO (time to recover): seconds for node/AZ/region loss (automatic). For restore-from-backup, RTO is dominated by data volume and storage throughput — size and test it.

12.3 Cross-cluster options (Enterprise)

- Physical Cluster Replication (PCR): continuously replicates a primary cluster to a standby cluster for fast promotion on catastrophic primary loss.
- Changefeeds / logical replication: stream data to another system for heterogeneous DR or analytics offload.

> **BEST PRACTICE**
> Write and rehearse a runbook for the two events automation cannot fully solve: (1) restore-to-new-cluster from backups, and (2) standby promotion. Measure real RTO during a game-day so the number in your DR plan is observed, not assumed.

13. High Availability

High availability is CockroachDB's core value proposition. There is no primary/standby to fail over; every range is independently highly available through Raft, and the cluster continuously repairs itself.

13.1 How writes stay consistent and available

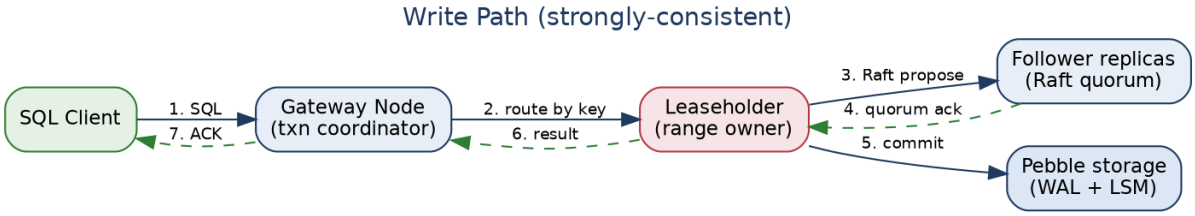


Figure 13.1 — Write path: quorum commit keeps data consistent even if a replica is down.

Mermaid source (editable / re-renderable):

```
flowchart LR
  c[SQL Client] --> gw[Gateway Node]
  gw --> lh[Leaseholder]
  lh --> rf[Follower replicas]
  rf -. quorum ack .-> lh
  lh --> st[(Pebble storage)]
  lh -. result .-> gw -. ACK .-> c
```

13.2 Automatic failover & self-healing

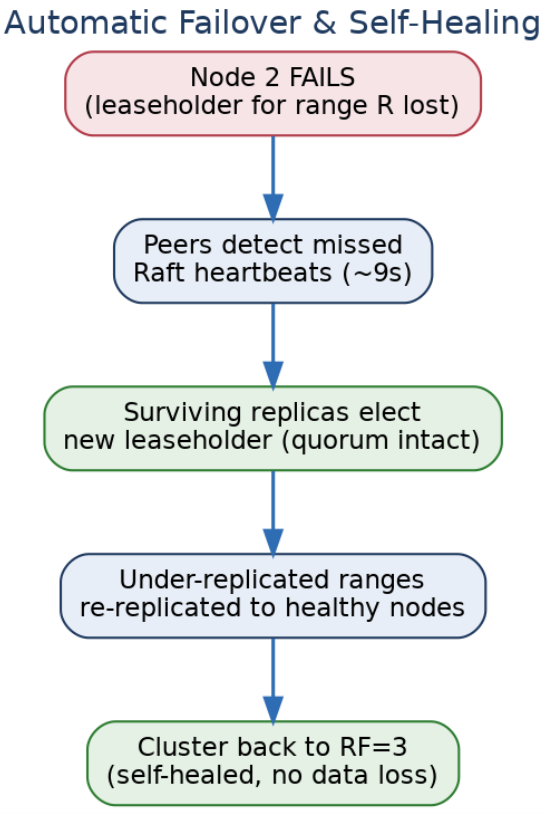


Figure 13.2 — Node failure triggers re-election and re-replication with no operator action.

Mermaid source (editable / re-renderable):

```
flowchart TD
  f["Node fails (leaseholder lost)"] --> d["Peers miss Raft heartbeats
(~9s)"]
  d --> e["Survivors elect new leaseholder"]
  e --> g["Under-replicated ranges re-replicate"]
  g --> h["Back to RF=3 — self-healed"]
```

13.3 Multi-region & survival goals

Multi-region tables let you choose a survival goal and a table locality that together determine placement and the latency/resilience trade-off.

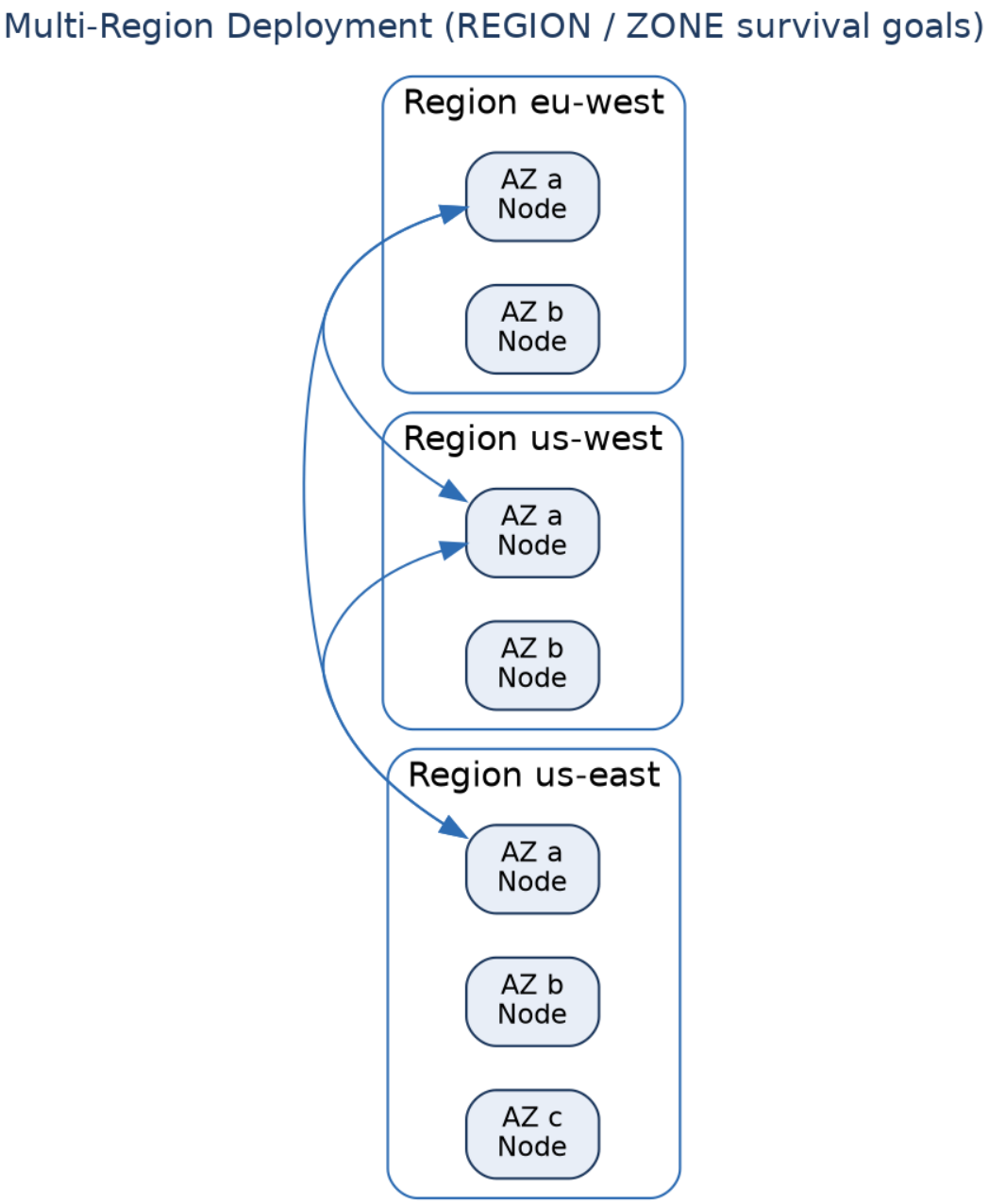


Figure 13.3 — Nodes spread across regions/AZs for region-level survival.

Mermaid source (editable / re-renderable):

```
flowchart LR
  subgraph us-east
   a1[Node] & a2[Node] & a3[Node]
  end
  subgraph us-west
   b1[Node] & b2[Node]
  end
  subgraph eu-west
   c1[Node] & c2[Node]
  end
  a1 <--> b1
  b1 <--> c1
  a1 <--> c1
```

Concept	Options	Effect
Survival goal	`ZONE` (default) / `REGION`	ZONE survives AZ loss; REGION survives a full region loss (needs ≥ 3 regions)
Table locality	`REGIONAL BY TABLE`	All rows homed in one region — fast local writes there
Table locality	`REGIONAL BY ROW`	Each row homed in its own region — data domiciling / residency
Table locality	`GLOBAL`	Low-latency reads everywhere (for reference/lookup data)

> **NOTE**
> REGION survival requires at least three regions so that losing one still leaves a quorum. Choosing REGIONAL BY ROW is also the primary mechanism for GDPR-style data residency — rows physically stay in their home region.

14. Troubleshooting

A structured approach: confirm cluster health, locate the failing component (node, range, or statement), then drill into logs and internal tables. The DB Console and `crdb_internal` virtual tables are the primary tools.

14.1 First-line diagnostics

```
# Node liveness & membership
cockroach node status --certs-dir=certs --host=<node>
cockroach node status --ranges --stats --certs-dir=certs --host=<node>

# Problem ranges (unavailable / under-replicated / slow)
# DB Console → Advanced Debug → Problem Ranges, or:
SELECT * FROM crdb_internal.ranges WHERE unavailable = true;

# Cluster-wide hot ranges
SELECT * FROM crdb_internal.hot_ranges ORDER BY qps DESC LIMIT 20;
```

14.2 Common investigation paths

Symptom	Where to look	Likely cause
High p99 latency	SQL Activity; `EXPLAIN ANALYZE`	Missing index, hot range, contention, cross-region hops
`40001` retry errors	App logs; contention page	Write/write contention — shorten txns, retry with backoff
Under-replicated ranges	DB Console → Replication	Down node, slow disk, decommission in progress
Unavailable ranges	Problem Ranges	Lost quorum — too many replicas down in one domain
Node marked dead	HEALTH log; `node status`	Clock skew, disk stall, network partition, OOM
Disk filling	Node map; `capacity_available`	GC TTL too long, backup temp files, uneven rebalancing

14.3 Deep-dive tools

- `crdb_internal` schema: `ranges`, `hot_ranges`, `cluster_queries`, `node_statement_statistics`, `gossip_liveness`.
- `cockroach debug zip --certs-dir=certs --host=<node> crdb.zip` — collects logs, schema, and internal state for support.
- DB Console → Advanced Debug → Hot Ranges / Problem Ranges / Runtime traces.

> **TIP**
> Run `cockroach debug zip` early when opening a support case or doing a post-mortem — it snapshots the state you will otherwise lose as the cluster self-heals.

15. Common Errors

Error / code	Meaning	Resolution
`40001` restart transaction	Serialization conflict under contention	Retry the whole txn with exponential backoff; shorten txns; reduce hot keys
`context deadline exceeded`	Operation timed out (often range unavailable or slow disk)	Check problem ranges, node liveness, and disk latency
`replica unavailable` / range has no quorum	Too many replicas of a range are down	Restore quorum: recover/replace down nodes; check AZ/region outage
`node is not live` / clock skew	Node ejected for exceeding max clock offset	Fix NTP/chrony; verify < 500 ms offset before restart
`store is full` / no space	Disk near capacity; writes blocked	Add capacity/nodes; lower GC TTL; ensure rebalancing; clear temp
`RETRY_WRITE_TOO_OLD`	Write raced ahead of a newer timestamp	Automatic internal retry usually resolves; reduce contention if frequent

Error / code	Meaning	Resolution
`connection refused` on 26257	Node down or firewall/LB misrouted	Verify process, port, TLS certs, and load-balancer health checks
`certificate has expired`	Node/CA/client cert past validity	Rotate certs and SIGHUP reload; automate expiry monitoring
`is not in the SELECT / GROUP BY`	PostgreSQL-strict SQL differences	Adjust SQL — CRDB follows Postgres grouping rules closely

BEST PRACTICE

Retryable errors (40001, RETRY_*) are a normal part of an optimistic-concurrency, serializable database — build a transaction retry loop into your data-access layer once, and most of these disappear from your on-call rotation.

16. FAQs

Q: Is CockroachDB compatible with PostgreSQL?

A: It implements the PostgreSQL wire protocol and a large subset of the dialect, so most Postgres drivers/ORMs work. It is not 100% — some extensions, stored-procedure features, and edge SQL differ. Test your workload rather than assuming full parity.

Q: Is CockroachDB open source?

A: No longer. It moved Apache → BSL (2019) → the proprietary, source-available CockroachDB Software License (Nov 2024). Source is viewable; self-hosted use needs a license key (a free tier exists for small orgs/individuals, with mandatory telemetry).

Q: What isolation level does it use?

A: Serializable by default — the strongest SQL isolation level. Read Committed is also available if you explicitly opt in. There are no dirty reads.

Q: How many nodes do I need?

A: Three is the practical minimum for resilience at RF=3 (survives one node loss). Spread across ≥ 3 fault domains for AZ survival, and ≥ 3 regions for region survival.

Q: Does it shard manually like MySQL?

A: No. Sharding (range splits), replica placement, and rebalancing are automatic. You influence placement declaratively via zone configs and multi-region SQL, but you never write shard logic.

Q: Can it replace my analytics warehouse?

A: Not for heavy OLAP scans. It targets OLTP/HTAP. Stream data out via changefeeds to a columnar warehouse for large analytical workloads.

Q: What storage engine does it use?

A: Pebble, an LSM-tree key-value engine written in Go (a RocksDB successor), with a write-ahead log per store.

Q: How does it keep clocks consistent?
A: It uses Hybrid Logical Clocks (HLC) and a configurable maximum clock offset. Nodes exceeding the offset are removed, so accurate NTP/chrony is mandatory.

Q: What happens during an upgrade?
A: Rolling, node-by-node, with no downtime. A finalize step activates new-version features; until finalized you can roll back within one major version.

Q: How do I connect existing apps?
A: Point your Postgres driver at the load balancer on port 26257 with TLS. Connection string looks like a standard `postgresql://` URL.

17. Interview Questions

Grouped from fundamentals to senior/architect level. Concise model answers are given for quick revision.

17.1 Fundamentals

Q. What is CockroachDB and what problem does it solve?
A. A distributed SQL database giving horizontal scale and strong consistency with automatic replication and survivability — removing the manual sharding and failover of traditional RDBMS while keeping SQL and ACID.

Q. Explain a range, a replica, and a leaseholder.
A. A range is a ~512 MiB contiguous slice of the keyspace; replicas are its copies (default 3) on different nodes; the leaseholder is the replica that serves reads and sequences writes for that range.

Q. How does CockroachDB achieve consistency?
A. Each range's replicas form a Raft group; a write commits only after a quorum acknowledges it, giving serializable, strongly-consistent results.

17.2 Architecture & internals

Q. Walk through the five layers.
A. SQL (plan/execute) → Transactional (MVCC, serializable ACID) → Distribution (sorted keyspace, ranges, routing) → Replication (Raft, leaseholders) → Storage (Pebble LSM).

Q. How are writes made durable and available?
A. The leaseholder proposes the write via Raft; once a quorum of replicas persists it to their WAL, it commits — so a minority of replicas can be down with no loss.

Q. Why are Hybrid Logical Clocks needed?
A. To order transactions across nodes without perfect global time; combined with a bounded max clock offset they give consistent timestamps for MVCC and serializability.

Q. How does the cluster self-heal after a node loss?
A. Peers detect missed heartbeats, survivors re-elect leaseholders, and under-replicated ranges are re-replicated onto healthy nodes back to the desired RF — automatically.

17.3 Operations & tuning

Q. How do you avoid write hotspots?
A. Avoid monotonic primary keys; use UUIDs or hash-sharded indexes to spread inserts across ranges.

Q. What causes 40001 errors and how do you handle them?
A. Serialization conflicts under contention. Handle with a client-side retry loop with exponential backoff, shorten transactions, and reduce contended keys.

Q. What are follower reads and when do you use them?
A. Slightly-stale reads served by the nearest replica via AS OF SYSTEM TIME, used to reduce latency and offload leaseholders for read-heavy, staleness-tolerant queries.

Q. How do you back up and restore with minimal data loss?
A. Scheduled full + frequent incrementals WITH revision_history to object storage; restore with AS OF SYSTEM TIME for point-in-time recovery; test restores regularly.

17.4 Architect / scenario

Q. Design a globally-available app that keeps EU data in the EU.
A. Multi-region database with regions incl. eu-west; SURVIVE REGION FAILURE; REGIONAL BY ROW on customer tables to domicile EU rows in eu-west; GLOBAL tables for shared reference data; follower reads for local low-latency reads.

Q. 3 vs 5 replicas — trade-offs?
A. 3 tolerates 1 failure with least overhead; 5 tolerates 2 and smooths tail latency during a single failure, at higher storage/write cost. Use 5 for the most critical tables via zone config.

Q. CockroachDB vs a traditional primary/replica RDBMS?
A. CockroachDB offers automatic sharding, no single write primary, synchronous quorum replication, and self-healing HA/geo-distribution; traditional setups need manual sharding, async replicas, and orchestrated failover.

18. Real-world Use Cases

Domain	Use case	Why CockroachDB fits
Financial services	Ledgers, payments, order books	Serializable ACID + zero-loss HA; audit logging for SOX/PCI-DSS
Retail / e-commerce	Carts, inventory, checkout at scale	Horizontal write scale; survives node/AZ loss during peak sales
SaaS / multi-tenant	Per-tenant data with residency	REGIONAL BY ROW domiciling; scale-out without re-sharding
Gaming	Global player state & leaderboards	Low-latency local reads (follower/GLOBAL tables); geo-distribution
Identity / auth	Sessions, tokens, entitlements	Always-on availability; strong consistency for security state

Domain	Use case	Why CockroachDB fits
IoT / telemetry	High-ingest event stores	Hash-sharded indexes spread write load; changefeeds to analytics
Logistics	Real-time tracking & routing	Multi-region writes with regional homing for locality

NOTE
Common thread: applications that outgrow a single-primary RDBMS and cannot accept downtime or data loss, especially with geographic or regulatory distribution requirements.

19. Best Practices

19.1 Deployment

- Run ≥ 3 nodes across ≥ 3 fault domains; use odd replication factors (3 or 5).
- Dedicated local SSD/NVMe per node; one store per physical device; keep stores ≤ ~2.5 TB.
- Set `--locality` accurately (region/zone) — placement and survival goals depend on it.
- Front the cluster with an L4 load balancer using `/health?ready=1` checks.

19.2 Schema & queries

- Avoid monotonic keys; use UUID or hash-sharded indexes for high-insert tables.
- Keep transactions short; implement a retry loop for 40001/RETRY_* errors.
- Use covering (STORING) indexes; verify plans with `EXPLAIN ANALYZE`.
- Use follower reads for staleness-tolerant, read-heavy paths.

19.3 Security & compliance

- Always run secure (TLS + cert auth); never expose insecure mode in production.
- RBAC least privilege; reserve root/admin for break-glass; audit sensitive tables.
- Encryption at rest (Enterprise or encrypted volumes); secrets in a manager, never in code.
- Automate certificate expiry monitoring and rotation.

19.4 Operations

- Monitor liveness, under-/unavailable ranges, p99 latency, capacity, and clock offset with alerts.
- Scheduled full + incremental backups WITH revision_history; test restores routinely.
- Upgrade node-by-node; validate on staging; know the rollback/finalize model.
- Run periodic game-days for node/AZ/region failure and restore-from-backup to measure real RTO/RPO.

BEST PRACTICE
The single highest-leverage habit: a tested backup + a rehearsed restore. CockroachDB automates most failure modes, so the disasters that actually hurt are the ones automation can’t undo — accidental DROPs, bad migrations, and total-cluster loss.

20. References

Authoritative sources for deeper study and current details (versions, licensing, and features evolve — always confirm against the official docs).

- Official documentation — https://www.cockroachlabs.com/docs/
- Architecture overview — https://www.cockroachlabs.com/docs/stable/architecture/overview
- Licensing FAQs & enterprise license — https://www.cockroachlabs.com/docs/stable/licensing-faqs
- Releases & support policy — https://www.cockroachlabs.com/docs/releases/
- Backup & restore — https://www.cockroachlabs.com/docs/stable/backup-and-restore-overview
- Multi-region capabilities — https://www.cockroachlabs.com/docs/stable/multiregion-overview
- Performance best practices — https://www.cockroachlabs.com/docs/stable/performance-best-practices-overview
- Monitoring & alerting — https://www.cockroachlabs.com/docs/stable/monitoring-and-alerting
- Production checklist — https://www.cockroachlabs.com/docs/stable/recommended-production-settings
- GitHub repository — https://github.com/cockroachdb/cockroach

End of document